Autômatos finitos: ler e traçar

A máquina que só sabe onde está

Encontro 2 | 05 de agosto

Linguagens Formais e Compiladores

Onde paramos

- Vimos as fases de um compilador, e que cada tipo de erro é pego por uma fase diferente.

Onde paramos

- Vimos as fases de um compilador, e que cada tipo de erro é pego por uma fase diferente.
- Combinamos sete palavras: símbolo, alfabeto, palavra, linguagem, gramática, padrão, lexema, token.

Onde paramos

- Vimos as fases de um compilador, e que cada tipo de erro é pego por uma fase diferente.
- Combinamos sete palavras: símbolo, alfabeto, palavra, linguagem, gramática, padrão, lexema, token.
- E demos uma primeira olhada em autômatos finitos, no fim de três horas.

Onde paramos

- Vimos as fases de um compilador, e que cada tipo de erro é pego por uma fase diferente.
- Combinamos sete palavras: símbolo, alfabeto, palavra, linguagem, gramática, padrão, lexema, token.
- E demos uma primeira olhada em autômatos finitos, no fim de três horas.

O QUE VAMOS FAZER HOJE

Voltar aos autômatos com calma, e desta vez **com as mãos**. Hoje é só ler e traçar. Construir autômato a partir de um problema é o próximo encontro.

Onde paramos

- Vimos as fases de um compilador, e que cada tipo de erro é pego por uma fase diferente.
- Combinamos sete palavras: símbolo, alfabeto, palavra, linguagem, gramática, padrão, lexema, token.
- E demos uma primeira olhada em autômatos finitos, no fim de três horas.

O QUE VAMOS FAZER HOJE

Voltar aos autômatos com calma, e desta vez **com as mãos**. Hoje é só ler e traçar. Construir autômato a partir de um problema é o próximo encontro.

Se você nunca viu autômato antes, esta aula começa do zero.

O que você vai saber fazer ao sair

- Percorrer uma palavra num diagrama e dizer se ela é aceita.

O que você vai saber fazer ao sair

- Percorrer uma palavra num diagrama e dizer se ela é aceita.
- Distinguir as duas formas de uma palavra ser rejeitada.

O que você vai saber fazer ao sair

- Percorrer uma palavra num diagrama e dizer se ela é aceita.
- Distinguir as duas formas de uma palavra ser rejeitada.
- Converter diagrama em tabela e tabela em diagrama.

O que você vai saber fazer ao sair

- Percorrer uma palavra num diagrama e dizer se ela é aceita.
- Distinguir as duas formas de uma palavra ser rejeitada.
- Converter diagrama em tabela e tabela em diagrama.
- Escrever a quíntupla olhando um desenho, e desenhar olhando uma quíntupla.

O que você vai saber fazer ao sair

- Percorrer uma palavra num diagrama e dizer se ela é aceita.
- Distinguir as duas formas de uma palavra ser rejeitada.
- Converter diagrama em tabela e tabela em diagrama.
- Escrever a quíntupla olhando um desenho, e desenhar olhando uma quíntupla.
- Decidir se a palavra vazia é aceita, e justificar.

O que você vai saber fazer ao sair

- Percorrer uma palavra num diagrama e dizer se ela é aceita.
- Distinguir as duas formas de uma palavra ser rejeitada.
- Converter diagrama em tabela e tabela em diagrama.
- Escrever a quíntupla olhando um desenho, e desenhar olhando uma quíntupla.
- Decidir se a palavra vazia é aceita, e justificar.
- Completar um autômato que está incompleto.

O que você vai saber fazer ao sair

- Percorrer uma palavra num diagrama e dizer se ela é aceita.
- Distinguir as duas formas de uma palavra ser rejeitada.
- Converter diagrama em tabela e tabela em diagrama.
- Escrever a quíntupla olhando um desenho, e desenhar olhando uma quíntupla.
- Decidir se a palavra vazia é aceita, e justificar.
- Completar um autômato que está incompleto.
- E o mais difícil: olhar um autômato e descrever em português qual linguagem ele reconhece.

O que você vai saber fazer ao sair

- Percorrer uma palavra num diagrama e dizer se ela é aceita.
- Distinguir as duas formas de uma palavra ser rejeitada.
- Converter diagrama em tabela e tabela em diagrama.
- Escrever a quíntupla olhando um desenho, e desenhar olhando uma quíntupla.
- Decidir se a palavra vazia é aceita, e justificar.
- Completar um autômato que está incompleto.
- E o mais difícil: olhar um autômato e descrever em português qual linguagem ele reconhece.

O último item é o objetivo real. Os outros seis são o caminho.

1 Comecemos por um interruptor

Uma pergunta boba

interruptor de luz

começa apagada

você aperta **um número qualquer** de vezes

Uma pergunta boba

interruptor de luz

começa apagada

você aperta **um número qualquer** de vezes

A luz está acesa?

Uma pergunta boba

interruptor de luz

começa apagada

você aperta **um número qualquer** de vezes

A luz está acesa?

Responda mentalmente. E repare no que você *não* precisou saber.

O que você não precisou saber

- Não precisou saber quantas vezes apertou.

O que você não precisou saber

- Não precisou saber quantas vezes apertou.
- Não precisou saber quando apertou.

O que você não precisou saber

- Não precisou saber quantas vezes apertou.
- Não precisou saber quando apertou.
- Não precisou de nada além de uma informação: está acesa ou apagada agora.

O que você não precisou saber

- Não precisou saber quantas vezes apertou.
- Não precisou saber quando apertou.
- Não precisou de nada além de uma informação: está acesa ou apagada agora.

Cinco apertos ou trezentos e um: dá no mesmo.

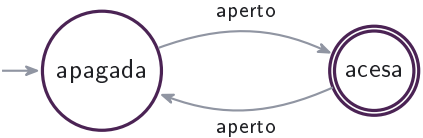
O que você não precisou saber

- Não precisou saber quantas vezes apertou.
- Não precisou saber quando apertou.
- Não precisou de nada além de uma informação: está acesa ou apagada agora.

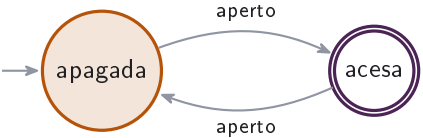
Cinco apertos ou trezentos e um: dá no mesmo.

O interruptor não conta. Ele só lembra em qual das duas situações está.

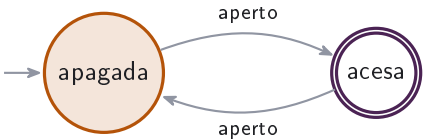
O interruptor, desenhado



O interruptor, desenhado



O interruptor, desenhado



JÁ ESTÁ TUDO AQUI

Os círculos são as **duas situações possíveis**. As flechas dizem o que acontece a cada aperto. A flecha que vem do nada marca onde se começa. O círculo duplo marca a situação que responde *sim* à pergunta.

Os cinco elementos que já estavam ali

no interruptor	**o nome que vamos usar**
apagada, acesa	os **estados**
um aperto	um **símbolo** da entrada
a sequência de apertos	uma **palavra**
começa apagada	o **estado inicial**
acesa responde sim	o **estado de aceitação**

Os cinco elementos que já estavam ali

no interruptor	**o nome que vamos usar**
apagada, acesa	os **estados**
um aperto	um **símbolo** da entrada
a sequência de apertos	uma **palavra**
começa apagada	o **estado inicial**
acesa responde sim	o **estado de aceitação**

Não inventamos nada. Só demos nome ao que já existia.

E se a pergunta mudasse?

"Quantas vezes você apertou?"

E se a pergunta mudasse?

"Quantas vezes você apertou?"

- Agora dois estados não bastam.

E se a pergunta mudasse?

"Quantas vezes você apertou?"

- Agora dois estados não bastam.
- Para responder *quantas*, você precisaria de um estado para cada contagem possível.

E se a pergunta mudasse?

"Quantas vezes você apertou?"

- Agora dois estados não bastam.
- Para responder *quantas*, você precisaria de um estado para cada contagem possível.
- E as contagens possíveis não têm fim.

E se a pergunta mudasse?

"Quantas vezes você apertou?"

- Agora dois estados não bastam.
- Para responder *quantas*, você precisaria de um estado para cada contagem possível.
- E as contagens possíveis não têm fim.

GUARDE ESTA OBSERVAÇÃO

A máquina funciona porque a **pergunta** cabe em memória fixa. Trocando a pergunta, a mesma máquina deixa de servir. Isso vai render um teorema.

2

Você já escreveu isso

Um código sem novidade

```
estado = 'apagada'

for _ in apertos:
    if estado == 'apagada':
        estado = 'acesa'
    else:
        estado = 'apagada'

print(estado == 'acesa')
```

Um código sem novidade

```
estado = 'apagada'

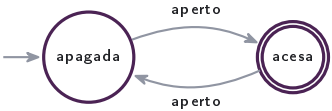
for _ in apertos:
    if estado == 'apagada':
        estado = 'acesa'
    else:
        estado = 'apagada'

print(estado == 'acesa')
```

Quem já escreveu um laço com essa forma?

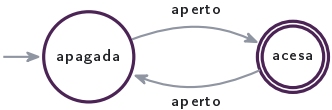
O código e o desenho

```
estado = 'apagada'
for _ in apertos:
    if estado == 'apagada':
        estado = 'acesa'
    else:
        estado = 'apagada'
print(estado == 'acesa')
```



O código e o desenho

```
estado = 'apagada'
for _ in apertos:
    if estado == 'apagada':
        estado = 'acesa'
    else:
        estado = 'apagada'
print(estado == 'acesa')
```



O desenho não é um conceito novo.
É uma notação para este código.

E a terceira forma: a tabela

```
delta = {('apagada', 'aperto'): 'acesa',
         ('acesa', 'aperto'): 'apagada'}

estado = 'apagada'
for s in apertos:
    estado = delta[(estado, s)] # consulta, atualiza, repete
print(estado == 'acesa')
```

E a terceira forma: a tabela

```
delta = {('apagada', 'aperto'): 'acesa',
         ('acesa', 'aperto'): 'apagada'}

estado = 'apagada'
for s in apertos:
    estado = delta[(estado, s)] # consulta, atualiza, repete
print(estado == 'acesa')
```

OBSERVE

O laço encolheu para **uma linha**. Toda a lógica saiu do código e entrou na tabela. Trocar de autômato agora é trocar o dicionário, sem tocar no laço.

Três formas, uma máquina



Três formas, uma máquina



Converter entre as três, nos dois sentidos, é metade desta aula.

3
A definição formal

A quíntupla

Um autômato finito determinístico é uma quíntupla

$$M = (\Sigma,\ Q,\ \delta,\ q_0,\ F)$$

A quíntupla

Um autômato finito determinístico é uma quíntupla

$$M = (\Sigma,\ Q,\ \delta,\ q_0,\ F)$$

- Σ o **alfabeto**: os símbolos que podem aparecer

A quíntupla

Um autômato finito determinístico é uma quíntupla

$$M = (\Sigma,\ Q,\ \delta,\ q_0,\ F)$$

- Σ o **alfabeto**: os símbolos que podem aparecer
- Q o conjunto **finito** de estados

A quíntupla

Um autômato finito determinístico é uma quíntupla

$$M = (\Sigma,\ Q,\ \delta,\ q_0,\ F)$$

- Σ o **alfabeto**: os símbolos que podem aparecer
- Q o conjunto **finito** de estados
- δ a **função de transição**: as flechas

A quíntupla

Um autômato finito determinístico é uma quíntupla

$$M = (\Sigma,\ Q,\ \delta,\ q_0,\ F)$$

- Σ o **alfabeto**: os símbolos que podem aparecer
- Q o conjunto **finito** de estados
- δ a **função de transição**: as flechas
- q_0 o **estado inicial**

A quíntupla

Um autômato finito determinístico é uma quíntupla

$$M = (\Sigma,\ Q,\ \delta,\ q_0,\ F)$$

- Σ o **alfabeto**: os símbolos que podem aparecer
- Q o conjunto **finito** de estados
- δ a **função de transição**: as flechas
- q_0 o **estado inicial**
- F o conjunto de estados **de aceitação**

A quíntupla

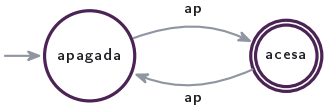
Um autômato finito determinístico é uma quíntupla

$$M = (\Sigma,\ Q,\ \delta,\ q_0,\ F)$$

- Σ o **alfabeto**: os símbolos que podem aparecer
- Q o conjunto **finito** de estados
- δ a **função de transição**: as flechas
- q_0 o **estado inicial**
- F o conjunto de estados **de aceitação**

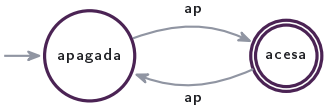
Cinco coisas. Faltando uma, o objeto não existe.

A quíntupla do interruptor

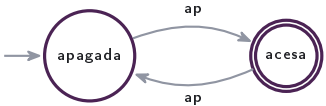


A quíntupla do interruptor

- $\Sigma = \{\texttt{ap}\}$

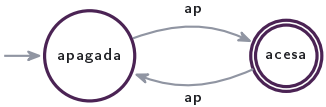


A quíntupla do interruptor



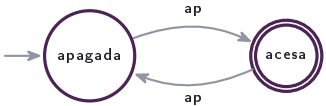
- $\Sigma = \{\texttt{ap}\}$
- $Q = \{\texttt{apagada}, \texttt{acesa}\}$

A quíntupla do interruptor



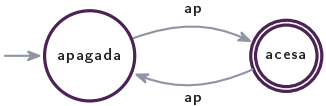
- $\Sigma = \{\texttt{ap}\}$
- $Q = \{\texttt{apagada},\ \texttt{acesa}\}$
- $\delta(\texttt{apagada}, \texttt{ap}) = \texttt{acesa}$
  $\delta(\texttt{acesa}, \texttt{ap}) = \texttt{apagada}$

A quíntupla do interruptor



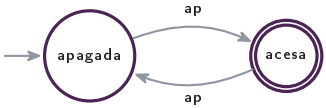
- $\Sigma = \{\texttt{ap}\}$
- $Q = \{\texttt{apagada},\ \texttt{acesa}\}$
- $\delta(\texttt{apagada}, \texttt{ap}) = \texttt{acesa}$
  $\delta(\texttt{acesa}, \texttt{ap}) = \texttt{apagada}$
- $q_0 = \texttt{apagada}$

A quíntupla do interruptor



- $\Sigma = \{\texttt{ap}\}$
- $Q = \{\texttt{apagada},\ \texttt{acesa}\}$
- $\delta(\texttt{apagada}, \texttt{ap}) = \texttt{acesa}$
  $\delta(\texttt{acesa}, \texttt{ap}) = \texttt{apagada}$
- $q_0 = \texttt{apagada}$
- $F = \{\texttt{acesa}\}$

A quíntupla do interruptor



- $\Sigma = \{\texttt{ap}\}$
- $Q = \{\texttt{apagada},\ \texttt{acesa}\}$
- $\delta(\texttt{apagada}, \texttt{ap}) = \texttt{acesa}$
  $\delta(\texttt{acesa}, \texttt{ap}) = \texttt{apagada}$
- $q_0 = \texttt{apagada}$
- $F = \{\texttt{acesa}\}$

A quíntupla é um inventário do desenho. Nada mais.

Duas palavras merecem atenção

- Q é finito. É a frase do interruptor, escrita em matemática: a máquina tem um número fixo de situações possíveis, e não pode inventar mais.

Duas palavras merecem atenção

- Q é finito. É a frase do interruptor, escrita em matemática: a máquina tem um número fixo de situações possíveis, e não pode inventar mais.
- δ é uma função. Para cada par estado e símbolo existe **no máximo um** destino.

Duas palavras merecem atenção

- Q é finito. É a frase do interruptor, escrita em matemática: a máquina tem um número fixo de situações possíveis, e não pode inventar mais.
- δ é uma função. Para cada par estado e símbolo existe **no máximo um** destino.

A SEGUNDA É O SIGNIFICADO DE "DETERMINÍSTICO"

Determinístico quer dizer exatamente isso: sem escolha. Diante de um estado e um símbolo, o destino está decidido.

O que determinístico NÃO significa

a máquina não faz	**a máquina faz**
tentar um caminho e voltar atrás	seguir a única flecha que existe
testar alternativas em paralelo	ler cada símbolo uma vez
olhar adiante para decidir	decidir com o estado atual e o símbolo atual

O que determinístico NÃO significa

a máquina não faz	a máquina faz
tentar um caminho e voltar atrás	seguir a única flecha que existe
testar alternativas em paralelo	ler cada símbolo uma vez
olhar adiante para decidir	decidir com o estado atual e o símbolo atual

POR QUE ESTOU INSISTINDO

É a intuição errada mais comum, e atrapalha duas vezes: agora, ao traçar palavras; e em três aulas, quando aparecer o autômato *não*-determinístico. Quem já achava que o AFD tentava caminhos não vai entender o que mudou.

Quando a máquina aceita

1. Comece em q_0.

Quando a máquina aceita

1. Comece em q_0.
2. Leia o próximo símbolo e siga a flecha correspondente.

Quando a máquina aceita

1. Comece em q_0.
2. Leia o próximo símbolo e siga a flecha correspondente.
3. Se não existir flecha para aquele símbolo, rejeite.

Quando a máquina aceita

1. Comece em q_0.
2. Leia o próximo símbolo e siga a flecha correspondente.
3. Se não existir flecha para aquele símbolo, rejeite.
4. Quando a palavra acabar, olhe onde você parou.

Quando a máquina aceita

1. Comece em q_0.
2. Leia o próximo símbolo e siga a flecha correspondente.
3. Se não existir flecha para aquele símbolo, rejeite.
4. Quando a palavra acabar, olhe onde você parou.

M aceita a palavra se esse estado pertence a F.

Quando a máquina aceita

1. Comece em q_0.
2. Leia o próximo símbolo e siga a flecha correspondente.
3. Se não existir flecha para aquele símbolo, rejeite.
4. Quando a palavra acabar, olhe onde você parou.

M aceita a palavra se esse estado pertence a F.

O MAL-ENTENDIDO DO CÍRCULO DUPLO

Círculo duplo **não** significa "pare aqui". Significa "se a palavra terminou aqui, aceite". A máquina pode sair de um estado de aceitação e voltar, quantas vezes quiser.

A linguagem reconhecida

$L(M)$ é o conjunto de *todas* as palavras que M aceita.

A linguagem reconhecida

$L(M)$ é o conjunto de *todas* as palavras que M aceita.

- Um autômato não aceita uma palavra. Ele define um conjunto.

A linguagem reconhecida

$L(M)$ é o conjunto de *todas* as palavras que M aceita.

- Um autômato não aceita uma palavra. Ele define um conjunto.
- No interruptor: $L(M)$ são as sequências de aperto em quantidade ímpar.

A linguagem reconhecida

$L(M)$ é o conjunto de *todas* as palavras que M aceita.

- Um autômato não aceita uma palavra. Ele define um conjunto.
- No interruptor: $L(M)$ são as sequências de aperto em quantidade ímpar.
- São infinitas palavras, definidas por dois círculos e duas flechas.

A linguagem reconhecida

$L(M)$ é o conjunto de *todas* as palavras que M aceita.

- Um autômato não aceita uma palavra. Ele define um conjunto.
- No interruptor: $L(M)$ são as sequências de aperto em quantidade ímpar.
- São infinitas palavras, definidas por dois círculos e duas flechas.

Voltamos ao glossário da aula 1: linguagem é conjunto de palavras.

Páginas de referência

Três páginas para guardar. Elas voltam em toda aula até o fim do semestre.

Referência 1: como se lê cada símbolo

símbolo	lê-se	é
Σ	sigma	o alfabeto
Q	quê	os estados
δ	delta	a função de transição
q_0	q zero	o estado inicial
F	efe	os estados de aceitação
ε	épsilon	a palavra vazia
w	dáblio	uma palavra qualquer
M	eme	o autômato inteiro

símbolo	lê-se	é
$\in$	pertence a	estar no conjunto
$\notin$	não pertence a	estar fora
$\subseteq$	está contido em	ser subconjunto
$\emptyset$	conjunto vazio	sem elementos
$\iff$	se e somente se	nos dois sentidos
Σ^*	sigma estrela	todas as palavras
$\lvert w \rvert$	compr. de w	quantos símbolos
$\times$	vezes	produto cartesiano

Referência 1: como se lê cada símbolo

símbolo	lê-se	é
Σ	sigma	o alfabeto
Q	quê	os estados
δ	delta	a função de transição
q_0	q zero	o estado inicial
F	efe	os estados de aceitação
ε	épsilon	a palavra vazia
w	dáblio	uma palavra qualquer
M	eme	o autômato inteiro

símbolo	lê-se	é
$\in$	pertence a	estar no conjunto
$\notin$	não pertence a	estar fora
$\subseteq$	está contido em	ser subconjunto
$\emptyset$	conjunto vazio	sem elementos
$\iff$	se e somente se	nos dois sentidos
Σ^*	sigma estrela	todas as palavras
$\lvert w \rvert$	compr. de w	quantos símbolos
$\times$	vezes	produto cartesiano

Se você não sabe dizer em voz alta, não consegue discutir. Comece por aqui.

Referência 2: o vocabulário formal

termo	em símbolos	exemplo
alfabeto: conjunto finito de símbolos	Σ	$\Sigma = \{\mathtt{a}, \mathtt{b}\}$
palavra: sequência finita de símbolos	$w \in \Sigma^*$	$w = \mathtt{abba}$
comprimento da palavra	$\lvert w \rvert$	$\lvert \mathtt{abba} \rvert = 4$
palavra vazia	ε	$\lvert \varepsilon \rvert = 0$
todas as palavras sobre Σ	Σ^*	$\{\varepsilon, \mathtt{a}, \mathtt{b}, \mathtt{aa}, \dots\}$
linguagem: conjunto de palavras	$L \subseteq \Sigma^*$	$L = \{\mathtt{a}, \mathtt{aa}\}$
linguagem vazia	$\emptyset$	nenhuma palavra

Referência 3: as definições do autômato

o que é	a definição
autômato finito determinístico	$M = (\Sigma,\ Q,\ \delta,\ q_0,\ F)$
função de transição	$\delta : Q \times \Sigma \to Q$
estado inicial	$q_0 \in Q$
estados de aceitação	$F \subseteq Q$
linguagem reconhecida	$L(M) = \{\, w \in \Sigma^* \mid M \text{ aceita } w \,\}$
a palavra vazia é aceita	$\varepsilon \in L(M) \iff q_0 \in F$
autômato completo	δ definida em *todo* $Q \times \Sigma$
linguagem regular	existe um AFD M com $L(M) = L$

Duas confusões que a referência evita

- ε **não** é o mesmo que $\emptyset$. O primeiro é uma palavra de comprimento zero; o segundo é um conjunto sem elementos. E $\{\varepsilon\}$ é diferente dos dois: é um conjunto com uma palavra dentro.

Duas confusões que a referência evita

- ε **não** é o mesmo que $\emptyset$. O primeiro é uma palavra de comprimento zero; o segundo é um conjunto sem elementos. E $\{\varepsilon\}$ é diferente dos dois: é um conjunto com uma palavra dentro.
- **Regular** não é elogio nem descrição vaga. Dizer que uma linguagem é regular significa exatamente uma coisa: existe um autômato finito que a reconhece.

Duas confusões que a referência evita

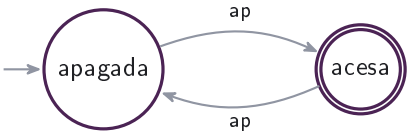
- ε **não** é o mesmo que $\emptyset$. O primeiro é uma palavra de comprimento zero; o segundo é um conjunto sem elementos. E $\{\varepsilon\}$ é diferente dos dois: é um conjunto com uma palavra dentro.
- **Regular** não é elogio nem descrição vaga. Dizer que uma linguagem é regular significa exatamente uma coisa: existe um autômato finito que a reconhece.

A SEGUNDA É A DEFINIÇÃO MAIS IMPORTANTE DO SEMESTRE

É ela que dá sentido à pergunta que vamos perseguir o curso todo. No encontro 7 provamos que *algumas linguagens não são regulares*, e é por isso que no encontro 10 precisaremos de uma máquina mais forte.

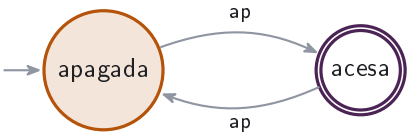
4
Traçar

Traçando no interruptor




```
ap ap ap
```

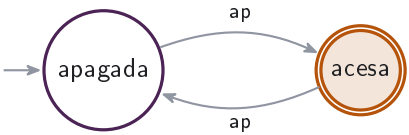
Traçando no interruptor



ap ap ap

apagada → acesa

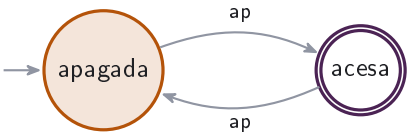
Traçando no interruptor



ap ap ap

acesa → apagada

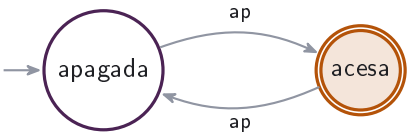
Traçando no interruptor



ap ap ap

apagada → acesa

Traçando no interruptor

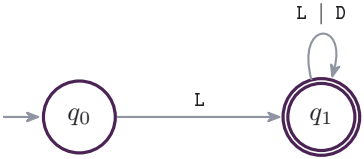


ap ap ap

parou em **acesa**, que pertence a F: **aceita**

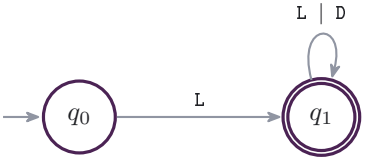
Um autômato de identificadores

Um identificador começa com letra e segue com letras ou dígitos.



Um autômato de identificadores

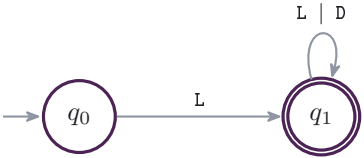
Um identificador começa com letra e segue com letras ou dígitos.



L vale por qualquer letra, D por qualquer dígito.

Um autômato de identificadores

Um identificador começa com letra e segue com letras ou dígitos.

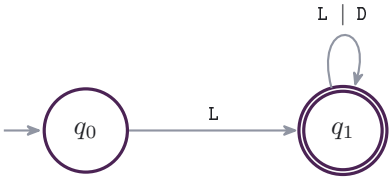


L vale por qualquer letra, D por qualquer dígito.

OBSERVE

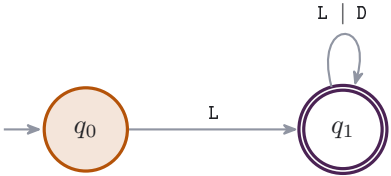
Repare no que **não** está desenhado: não existe flecha saindo de q_0 com D. Vamos usar isso.

Traçando L D D



L D D

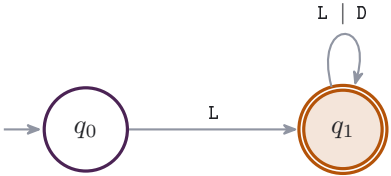
Traçando L D D



L D D

$q_0 \rightarrow q_1$

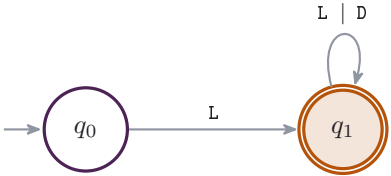
Traçando L D D



L D D

$q_1 \to q_1$ pelo laço

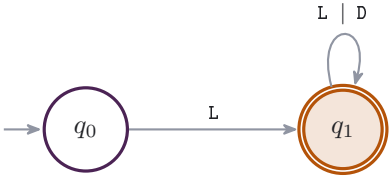
Traçando L D D



L D D

$q_1 \to q_1$ pelo laço

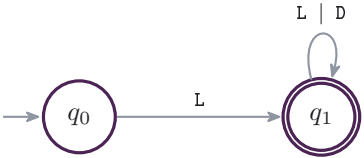
Traçando L D D



L D D

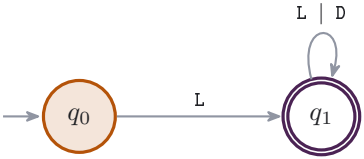
parou em $q_1 \in F$: **aceita**

Traçando D L



D L

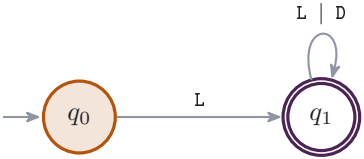
Traçando D L



D L

Estou em q_0 e li D. Não existe flecha. **A computação morre aqui.**

Traçando D L



D L

Estou em q_0 e li D. Não existe flecha. **A computação morre aqui.**

Rejeita. E nem chegou a olhar o L.

As duas formas de rejeitar

	a computação morre	**termina fora de F**
quando	falta flecha no meio do caminho	leu tudo, mas parou no estado errado
leu a palavra toda?	não	sim
exemplo	`D L` no identificador	`ap ap` no interruptor

As duas formas de rejeitar

	a computação morre	**termina fora de F**
quando	falta flecha no meio do caminho	leu tudo, mas parou no estado errado
leu a palavra toda?	não	sim
exemplo	`D L` no identificador	`ap ap` no interruptor

POR QUE SEPARAR AS DUAS

Dão o mesmo veredito por motivos diferentes. Quem não separa isso não entende a próxima seção, e escreve analisador léxico que trava em vez de reportar erro.

Sua vez

NO PAPEL, AGORA

No autômato de identificadores, trace estas cinco palavras. Para cada uma, escreva os estados visitados na ordem, e diga se aceita. Se rejeitar, diga **qual das duas formas**.

`L` `L L D` `D` `L D L D` `(palavra vazia)`

Sua vez

NO PAPEL, AGORA

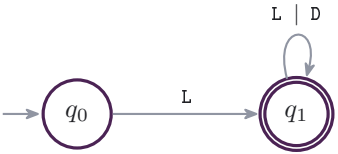
No autômato de identificadores, trace estas cinco palavras. Para cada uma, escreva os estados visitados na ordem, e diga se aceita. Se rejeitar, diga **qual das duas formas**.

`L` `L L D` `D` `L D L D` `(palavra vazia)`

A última é diferente das outras quatro. Guarde a dúvida: ela tem uma seção só para ela.

5
Diagrama e tabela

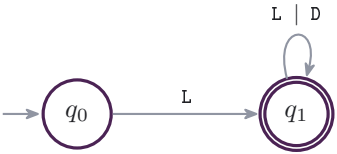
Do diagrama para a tabela



δ	L	D
$\rightarrow q_0$	q_1	
$*\, q_1$	q_1	q_1

$\rightarrow$ inicial $*$ de aceitação

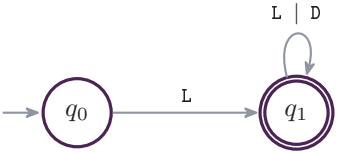
Do diagrama para a tabela



δ	L	D
$\rightarrow q_0$	q_1	—
$* q_1$	q_1	q_1

$\rightarrow$ inicial $*$ de aceitação

Do diagrama para a tabela



δ	L	D
$\rightarrow q_0$	q_1	—
$* q_1$	q_1	q_1

$\rightarrow$ inicial $*$ de aceitação

OBSERVE

A célula vazia não é esquecimento. Ela é a informação de que aquela transição não existe, e é o que rejeita `D L`.

Da tabela para o diagrama

NO PAPEL, AGORA

Desenhe o autômato correspondente a esta tabela. E depois descubra o que ele aceita.

δ	a	b
$\rightarrow s_0$	s_1	s_0
s_1	s_1	s_2
$* s_2$	s_1	s_0

Da tabela para o diagrama

NO PAPEL, AGORA

Desenhe o autômato correspondente a esta tabela. E depois descubra o que ele aceita.

δ	a	b
$\rightarrow s_0$	s_1	s_0
s_1	s_1	s_2
$*\, s_2$	s_1	s_0

Três estados, seis flechas, nenhuma célula vazia. Comece desenhando os círculos.

6

Completo e parcial

Duas maneiras de escrever o mesmo autômato

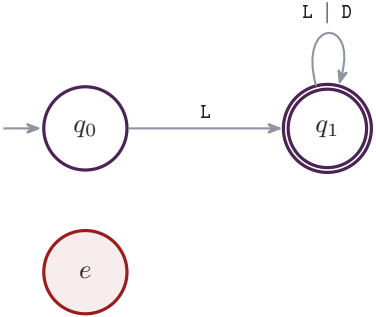
	parcial	**completo**
δ está definida	só onde há flecha	para *todo* par estado e símbolo
a tabela	tem células vazias	está toda preenchida
falta de flecha	significa rejeitar	não acontece

Duas maneiras de escrever o mesmo autômato

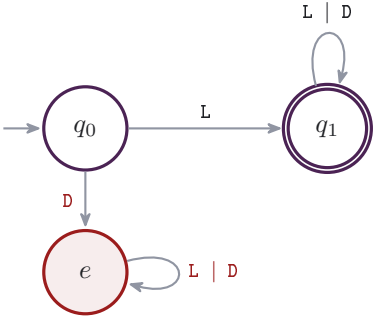
	parcial	**completo**
δ está definida	só onde há flecha	para *todo* par estado e símbolo
a tabela	tem células vazias	está toda preenchida
falta de flecha	significa rejeitar	não acontece

A forma parcial é a que se desenha. A completa é a que se implementa.

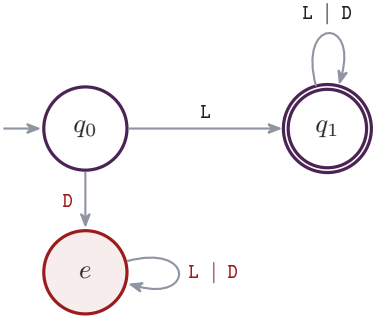
Completando: o estado de erro



Completando: o estado de erro



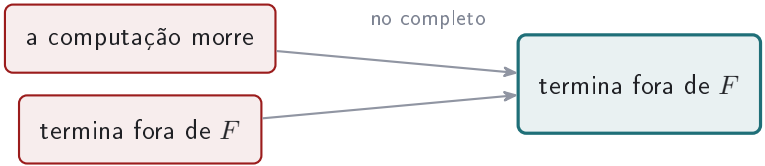
Completando: o estado de erro



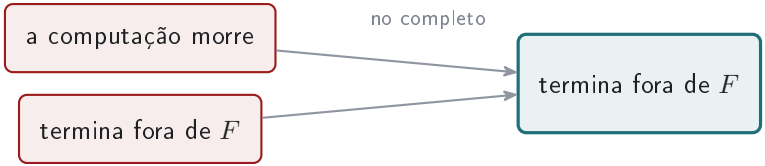
DUAS PROPRIEDADES OBRIGATÓRIAS

O estado de erro **não** pertence a F, e é **absorvente**: uma vez dentro, nada sai. Marcar e com círculo duplo por descuido faz o autômato aceitar tudo.

Por que alguém se dá esse trabalho

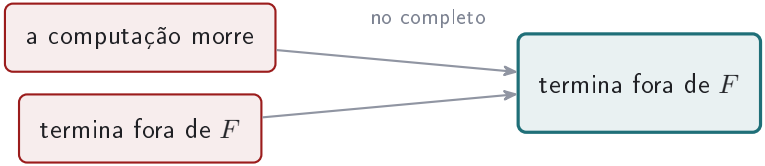


Por que alguém se dá esse trabalho



Completar o autômato transforma duas falhas em uma.

Por que alguém se dá esse trabalho



Completar o autômato transforma duas falhas em uma.

Ficar sem flecha nunca acontece. Falhar passa a ser sempre terminar num estado que não aceita.

E é por isso que o código fica simples

```
estado = q0
for simbolo in palavra:
    estado = delta[(estado, simbolo)] # nunca falta entrada
return estado in F
```

E é por isso que o código fica simples

```
estado = q0
for simbolo in palavra:
    estado = delta[(estado, simbolo)] # nunca falta entrada
return estado in F
```

SEM COMPLETAR, O LAÇO PRECISARIA DE UM TESTE A MAIS

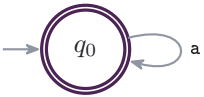
Com a tabela incompleta, cada passo teria que verificar se a transição existe antes de usá-la, e decidir o que fazer se não existir. Completar o autômato tira esse teste do laço.

7

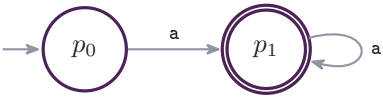
A palavra vazia

Dois autômatos quase idênticos

autômato 1

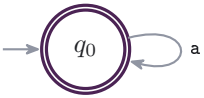


autômato 2

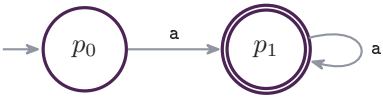


Dois autômatos quase idênticos

autômato 1



autômato 2



Qual a diferença entre as duas linguagens?

A diferença é uma palavra

	autômato 1	**autômato 2**
a, aa, aaa, ...	aceita	aceita
a palavra vazia ε	**aceita**	**rejeita**
$L(M)$	zero ou mais a	*uma* ou mais a

A diferença é uma palavra

	autômato 1	**autômato 2**
a, aa, aaa, ...	aceita	aceita
a palavra vazia ε	**aceita**	**rejeita**
$L(M)$	zero ou mais a	*uma* ou mais a

A única diferença no desenho é se q_0 tem círculo duplo.

A regra, de uma vez por todas

$$\varepsilon \in L(M) \quad \textbf{se e somente se} \quad q_0 \in F$$

A regra, de uma vez por todas

$$\varepsilon \in L(M) \quad \textbf{se e somente se} \quad q_0 \in F$$

POR QUE É ASSIM

Para consumir a palavra vazia, a máquina não lê símbolo nenhum. Então ela termina onde começou. Se o estado inicial aceita, ε é aceita; se não, não é.

A regra, de uma vez por todas

$$\varepsilon \in L(M) \quad \textbf{se e somente se} \quad q_0 \in F$$

POR QUE É ASSIM

Para consumir a palavra vazia, a máquina não lê símbolo nenhum. Então ela termina onde começou. Se o estado inicial aceita, ε é aceita; se não, não é.

No autômato de identificadores, q_0 não é de aceitação. Logo texto nenhum não é identificador.

Guarde este par

- Autômato 1 reconhece zero ou mais.

Guarde este par

- Autômato 1 reconhece zero ou mais.
- Autômato 2 reconhece uma ou mais.

Guarde este par

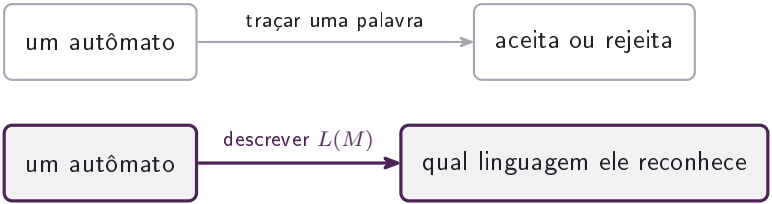
- Autômato 1 reconhece zero ou mais.
- Autômato 2 reconhece uma ou mais.

NO PRÓXIMO ENCONTRO ESSE PAR GANHA NOME

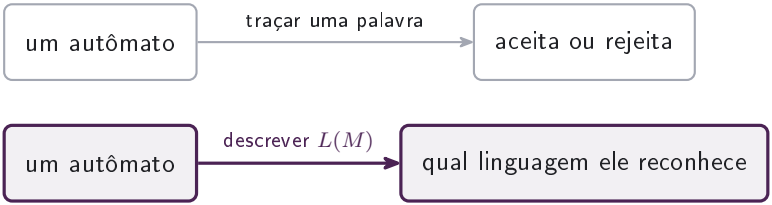
Vamos escrever expressões regulares, e a diferença entre a* e a+ é exatamente a diferença entre esses dois desenhos. Discutir ε hoje não é preciosismo: é adiantar trabalho.

8
O caminho inverso

A habilidade que importa

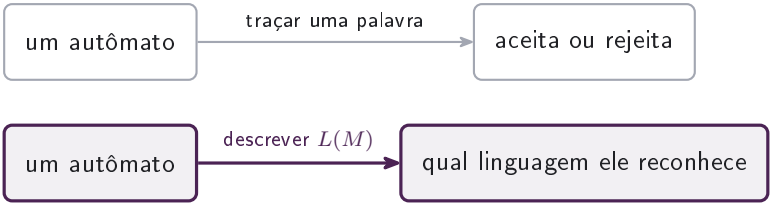


A habilidade que importa



Traçar é o meio. Descrever a linguagem é o fim.

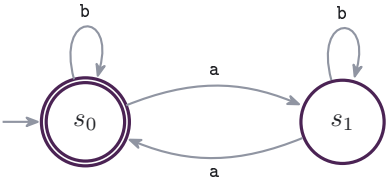
A habilidade que importa



Traçar é o meio. Descrever a linguagem é o fim.

Três autômatos a seguir. Descubram o que cada um aceita.

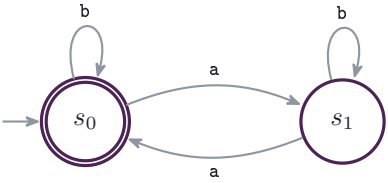
Autômato A



NO PAPEL, AGORA

Tracem b, ab, aab, ba e ε. Depois escrevam, em português, qual é $L(M)$.

Autômato A

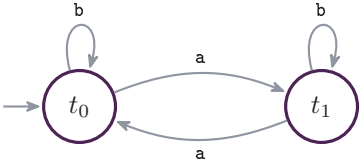


NO PAPEL, AGORA

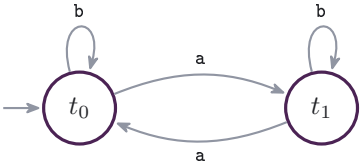
Tracem b, ab, aab, ba e ε. Depois escrevam, em português, qual é $L(M)$.

Dica: o que os estados estão contando?

Autômato B



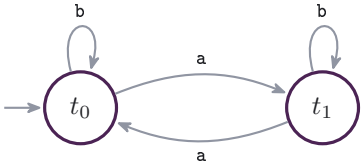
Autômato B



OBSERVE

É o autômato A com **uma** diferença. Qual? E o que ela faz com $L(M)$?

Autômato B

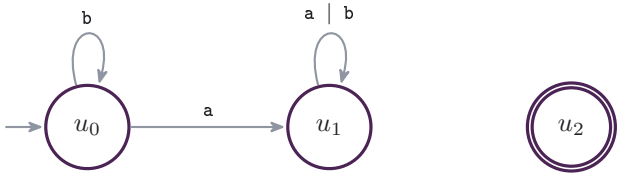


OBSERVE

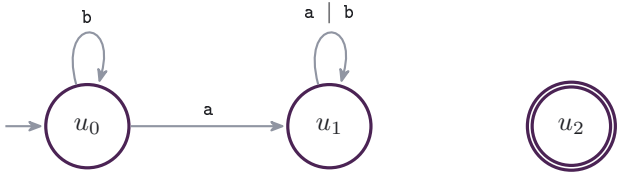
É o autômato A com **uma** diferença. Qual? E o que ela faz com $L(M)$?

$F = \emptyset$. **Nenhuma palavra é aceita:** $L(M) = \emptyset$.

Autômato C

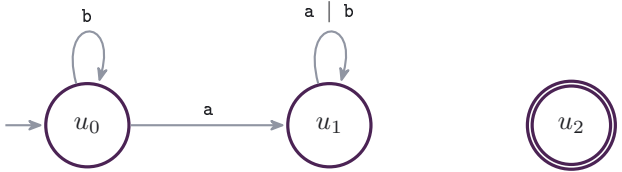


Autômato C



Existe estado de aceitação. Então $L(M)$ é vazia ou não?

Autômato C



Existe estado de aceitação. Então $L(M)$ é vazia ou não?

Vazia. Nenhuma flecha chega em u_2.

O que os três ensinaram

- **A** tinha estados que contavam algo: a paridade dos a.

O que os três ensinaram

- **A** tinha estados que contavam algo: a paridade dos a.
- **B** mostrou que sem estado de aceitação nada é aceito, e isso é um autômato legítimo. $L(M) = \emptyset$ é uma linguagem.

O que os três ensinaram

- **A** tinha estados que contavam algo: a paridade dos a.
- **B** mostrou que sem estado de aceitação nada é aceito, e isso é um autômato legítimo. $L(M) = \emptyset$ é uma linguagem.
- **C** mostrou o mais importante: ter círculo duplo não basta. Precisa haver caminho até ele.

O que os três ensinaram

- **A** tinha estados que contavam algo: a paridade dos a.
- **B** mostrou que sem estado de aceitação nada é aceito, e isso é um autômato legítimo. $L(M) = \emptyset$ é uma linguagem.
- **C** mostrou o mais importante: ter círculo duplo não basta. Precisa haver caminho até ele.

A LIÇÃO DO AUTÔMATO C

$L(M)$ não depende dos rótulos do desenho. Depende de quais estados são **alcançáveis** a partir de q_0. Um estado de aceitação inalcançável é decoração.

O que os três ensinaram

- **A** tinha estados que contavam algo: a paridade dos a.
- **B** mostrou que sem estado de aceitação nada é aceito, e isso é um autômato legítimo. $L(M) = \emptyset$ é uma linguagem.
- **C** mostrou o mais importante: ter círculo duplo não basta. Precisa haver caminho até ele.

A lição do autômato C

$L(M)$ não depende dos rótulos do desenho. Depende de quais estados são **alcançáveis** a partir de q_0. Um estado de aceitação inalcançável é decoração.

Guardem isso: em duas aulas vamos *remover* estados inalcançáveis de propósito.

9
Fechamento

O que fica desta aula

- Um autômato é um laço com uma variável de estado, escrito como desenho.

O que fica desta aula

- Um autômato é um laço com uma variável de estado, escrito como desenho.
- Três formas equivalentes: diagrama, tabela e quíntupla.

O que fica desta aula

- Um autômato é um laço com uma variável de estado, escrito como desenho.
- Três formas equivalentes: diagrama, tabela e quíntupla.
- Determinístico significa **sem escolha**: a máquina não tenta caminhos nem volta atrás.

O que fica desta aula

- Um autômato é um laço com uma variável de estado, escrito como desenho.
- Três formas equivalentes: diagrama, tabela e quíntupla.
- Determinístico significa **sem escolha**: a máquina não tenta caminhos nem volta atrás.
- Rejeitar acontece de duas maneiras, e completar o autômato reduz a uma.

O que fica desta aula

- Um autômato é um laço com uma variável de estado, escrito como desenho.
- Três formas equivalentes: diagrama, tabela e quíntupla.
- Determinístico significa **sem escolha**: a máquina não tenta caminhos nem volta atrás.
- Rejeitar acontece de duas maneiras, e completar o autômato reduz a uma.
- ε é aceita se e somente se $q_0 \in F$.

O que fica desta aula

- Um autômato é um laço com uma variável de estado, escrito como desenho.
- Três formas equivalentes: diagrama, tabela e quíntupla.
- Determinístico significa **sem escolha**: a máquina não tenta caminhos nem volta atrás.
- Rejeitar acontece de duas maneiras, e completar o autômato reduz a uma.
- ε é aceita se e somente se $q_0 \in F$.
- E $L(M)$ depende de **caminho**, não de rótulo.

O que fica desta aula

- Um autômato é um laço com uma variável de estado, escrito como desenho.
- Três formas equivalentes: diagrama, tabela e quíntupla.
- Determinístico significa **sem escolha**: a máquina não tenta caminhos nem volta atrás.
- Rejeitar acontece de duas maneiras, e completar o autômato reduz a uma.
- ε é aceita se e somente se $q_0 \in F$.
- E $L(M)$ depende de **caminho**, não de rótulo.

A frase para levar: o estado é tudo o que a máquina lembra.

Antes do próximo encontro

- **Terminem os três autômatos** do caminho inverso, com $L(M)$ escrito em português.

Antes do próximo encontro

- **Terminem os três autômatos** do caminho inverso, com $L(M)$ escrito em português.
- **Refaçam à mão**, sem olhar, o autômato de identificadores e a tabela dele.

Antes do próximo encontro

- **Terminem os três autômatos** do caminho inverso, com $L(M)$ escrito em português.
- **Refaçam à mão**, sem olhar, o autômato de identificadores e a tabela dele.
- **Continuem a atividade online**, se ainda não começaram.

Antes do próximo encontro

- **Terminem os três autômatos** do caminho inverso, com $L(M)$ escrito em português.
- **Refaçam à mão**, sem olhar, o autômato de identificadores e a tabela dele.
- **Continuem a atividade online**, se ainda não começaram.

NO PRÓXIMO ENCONTRO A MÃO TROCA DE LADO

Hoje vocês leram autômatos que eu dei. No encontro 3 vocês vão **construir** autômatos a partir de problemas em português, e a pergunta que guia tudo será: *o que cada estado precisa lembrar?*

O estado é tudo o que a máquina lembra.

Guardem esta frase. Em setembro ela vira uma demonstração de impossibilidade.